

**Ghulam Ishaq Khan Institute of Engineering Sciences &
Technology**

Faculty of Computer Science & Engineering



Learnify: Learn your way.

Ali Inayatullah 2021081

Raja Ata Ul Karim 2021528

Tehrim Ahmed 2021676

Advisor:

Engr. Ahsan Shah

Certificate of Approval

It is certified that the work presented in this report was performed by **Ali Inayatullah, Raja Ata Ul Karim, and Tehrim Ahmed** under the supervision of **Engr. Ahsan Shah**. The work is adequate and lies within the scope of the BS degree in Computer Science/Computer Engineering at Ghulam Ishaq Khan Institute of Engineering Sciences and Technology.

Engr. Ahsan Shah

Advisor

Prof. Dr. Qadeer Ul Hassan

Dean

SDGs Inclusion

For this Final Year Project, the Sustainable Development Goals that were aligned with Learnify were 4 (Quality Education), 9 (Industry, Innovation, and Infrastructure), and 10 (Reduced Inequalities). To align with SDG-4, which emphasizes Quality Education, the platform was created. The main purpose of the platform is to make quality education affordable for all, if there is an internet connection. To further improve the architecture, the RAG database was integrated with the system to provide more accurate knowledge.

For SDG-9, Learnify was designed to utilize and leverage industry practices and innovation and try to improve them, with the constraints of cost in mind. The platform utilizes a custom fine-tuned Llama 3.1 70B model, along with LangGraph for orchestrating multi-agent systems. Moreover, the integration of multiple different tools has further enhanced the user experience.

Addressing SDG-10, Learnify seeks to reduce inequalities in the education sector, with Pakistan facing a major disparity in the cost-to-quality ratio. The system has been designed to be as cost-efficient as possible while not reducing quality, attempting to eliminate language barriers by offering multilingual support (currently Urdu and English).

Engr. Ahsan Shah

Advisor

Prof. Dr. Qadeer Ul Hassan

Dean

Abstract

The traditional one-fits-all approach to Education is outdated and fails to address the diverse learning needs and different cognitive profiles of students. To fix this, we present Learnify, a multi-agentic and adaptive Educational Platform designed to personalize learning. Built using the LangGraph framework, Learnify leverages different agents for different calls, who coordinate around a fine-tuned Meta Llama 3.1 70B, designed to enhance reasoning.

Upon creation of account, a student is required to complete a Learning Style Diagnostic assessment, this enables the system to customize the teaching methodologies based on the individual preference of a student. Once a student enrolls to a course, they are taught on a chapter-by-chapter basis via AI-Generated Images, Text-based content, and MermaidJS Diagrams along with a multimodal learning environment. The generation of content is further bettered using Retrieval Augmented Generation (RAG) for each course. Additionally, Learnify offers multilingual support in English and Urdu.

By integrating an advanced Large Language Model (LLM), Agentic Graphs, Personalized Diagnostics, and multimodal content generation, Learnify aims to change the educational system.

Acknowledgements

The research and development to deliver Learnify would not have been possible without the guidance and support of:

- **Engr. Ahsan Shah (Advisor):**

Thank you for your continuous support throughout the duration of the Final Year Project, your guidance regarding the initial idea and the research was critical to Learnify.

- **Our Panellists:**

Thank you for your constant guidance throughout the different stages of the Final Year Project, a lot of guidance was detrimental to the final demo of the project.

- **Hafiz Syed Muhammad Muslim Shah (Coordinator):**

Thank you for taking out time to assist us and countless other Final Year Projects throughout the duration of this journey.

We are also grateful for other esteemed members of the Faculty of Computer Science for taking out time to help us in our problems, queries, and more.

Table of Contents

Chapter I 9

1. Introduction 9

Chapter II 14

2. Literature Survey..... 14

Chapter III..... 21

3. Design 21

Chapter IV 43

1. Proposed Solution 43

Chapter V 49

1. Results and Evaluation..... 49

List of Figures

FIGURE 1: UML USE-CASE DIAGRAM	32
FIGURE 2: ENTITY-RELATIONSHIP DIAGRAM	34
FIGURE 3: ACTIVITY DIAGRAM	37
FIGURE 4: CONTEXT DIAGRAM	40

List of Tables

TABLE 1: PERFORMANCE METRICS FOR THE MODEL	50
TABLE 2: PREDICTED LEARNING STYLE DISTRIBUTION	50
TABLE 3: PERFORMANCE METRICS FOR EACH IMAGE-GEN MODEL	51
TABLE 4: PERFORMANCE METRICS FOR DIFFERENT METHODOLOGIES	52
TABLE 5: SUMMARY OF ALL RESULTS	53

Chapter I

1. Introduction

1.1. *Motivation and Problem Statement:*

Despite development in educational technology, most available e-learning tools are still quite narrow in flexibility and customization. Standard online courseware and LMS tend to present standardized content, and test timetables no matter what variations in learners there may be. The absence of flexibility can generate less engagement because students with differing skill levels or experiences might get the content as too easy, too hard, or off-focus. For instance, when a student is having trouble with a subject, a typical LMS may not offer extra practice or explanations specific to that student without instructor intervention. Educational research indicates that adaptive learning systems can enhance course completion rates and learner satisfaction, while inflexible platforms tend to experience greater dropout and disengagement. Another challenge is the siloed nature of educational resources and AI tools. Most platforms concentrate on one mode of content delivery or interaction (e.g., video lectures, quizzes, or chatbots) with minimal coupling of different learning modes. In the absence of an orchestrated system of AI agents, current platforms do not easily integrate structured knowledge bases, generative AI, and recommendation engines in a harmonized manner. This isolated design implies that a student may be given an explanation by one side of the system and then must go hunt for practice problems on their own, breaking the flow of learning. Combining different AI capabilities (e.g., search, generation, personalization) calls for an architecture that can string together several services in a cohesive manner.

Security and user management are also top-of-mind issues in contemporary educational systems. Protecting student data and ensuring secure access require robust authentication and authorization mechanisms. Many learning platforms may rely on basic session-based authentication, which can be less scalable or more vulnerable than token-based approaches. For instance, session cookies can be hijacked or expire unexpectedly, whereas JSON Web Tokens (JWTs) can be stored in a secure, stateless manner to support scalable sessions. Teachers also increasingly want functionality to create custom visual content on the fly; however, most traditional LMS solutions lack native functionality for dynamically creating diagrams or illustrative aids.

In short, there is a definite demand for an integrated, smart learning platform that can adjust to individual learners' needs, integrating multiple AI-based features, and upholding contemporary security measures. Learnify seeks to fill these gaps by integrating a multi-agent framework with sophisticated AI models, facilitating dynamic personalization, rich content creation, and secure user management under one platform.

1.2. *Objectives and Scope*

The main goal of the Learnify project is to develop and deploy a multi-agent, AI-based learning platform that can provide customized educational material. To accomplish this, the project will follow some specific goals, such as integrating sophisticated AI models with web technologies and offering adaptive learning features. These goals are:

Creating a modular multi-agent architecture, based on LangGraph, to manage different AI-powered tasks like content retrieval, generation, and user feedback.

Building a context-aware retrieval system with a Pinecone vector database to allow the platform to search and retrieve learning content based on user queries and history.

Optimization of a huge language model (DeepSeek R1 70B) and incorporating it through the OpenRouter API to develop adaptive text-based content, descriptions, and quizzes specific to students.

Incorporating the OpenAI Image Generation API to develop personalized representative diagrams or study aids based on learner requirements.

Developing an interactive user interface using Next.js and a scalable backend using Node.js APIs with seamless interaction and live updates.

Involving secure user authentication and authorization with JSON Web Tokens (JWT) to safeguard user information and enable scalable, stateless sessions.

Presenting the effectiveness of the platform in case studies or through testing (if available), confirming the enhancements in learner interaction and understanding.

The focus of Learnify is on building the core system and its AI features. It excludes content authorship by instructors or commercial deployment but instead seeks to develop a prototype platform. The project will focus on integrating AI agents for student-facing functionality and will utilize synthetic or open education content for demonstration. Future additions (beyond this project) might include connecting to real learning management systems or growing the content database, but these are beyond the scope here.

1.3. *Project Contributions:*

The Learnify project has certain significant contributions to make to the field of learning technology and AI-powered learning platforms. These include architectural innovations, application of AI methods, and innovative integration of emerging tools:

- **Multi-agent System Design:** A multi-agent system design (LangGraph) designed specifically for adaptive learning, demonstrating how different AI agents can be combined together to address different sides of the learning process.
- **Fine-Tuned LLM Integration:** Implementing a fine-tuned large language model (DeepSeek R1 8B) in the learning platform to show how powerful generative AI can generate personalized explanations and exams for students.
- **Retrieval-Augmented Generation:** Utilization of a retrieval-augmented generation (RAG) approach within a Pinecone vector database to pull available text information to assist AI-generated content in referencing a highly curated educational library of content.
- **Generative Visual Content:** Employing generative visual content with the OpenAI Image Generation API, demonstrating the capability to utilize dynamically generated pictures and charts to symbolize complicated concepts.
- **Full-Stack Implementation:** Full-stack implementation using Next.js, Node.js, PostgreSQL, and integration of AI components, offering a template for scalable and efficient AI-based educational portals.

Together, these contributions illustrate how state-of-the-art AI techniques can be used to enrich personalized learning and lay a basis for further research in multi-agent learning systems. By showcasing this complete example, Learnify hopes to advance the convergence of AI and web technologies in learning software.

Chapter II

2. Literature Survey

2.1. *Personalized and Adaptive Learning Systems:*

Personalized and adaptive learning systems seek to customize educational content and experiences to individual learners' needs. Personalized instruction has been found to enhance learning outcomes by meeting each student's strengths and weaknesses (*Smith, 2023*). Adaptive learning systems usually contain user modelling components that approximate a learner's knowledge state and adapt the difficulty, sequencing, or format of content accordingly (*Ndeslihan Ademi, 2020*). Early implementations are ITS that offer guided step-by-step problem solving, adaptive tests or quizzes, or hints based on a student's level of performance. These typically use Bayesian knowledge tracing or other student modelling methods to forecast mastery of a topic and personalize instruction. With AI and big data, newer platforms build on these concepts and leverage real-time analytics and machine learning to iteratively adjust the path of learning.

For example, a system can monitor response patterns to quizzes or task completion times to detect areas of missing knowledge and present review materials or remedial exercises pre-emptively. Another method is concept mapping or clustering: mapping content into graphs of knowledge and assigning students to peers or resources with comparable learning profiles. Modern adaptive systems typically integrate recommendation algorithms like those for e-commerce or media streaming to choose the next learning item from a catalogue of content according to the learner's profile (*Minn, 2020*). Fingerprinting of content and clustering learners with similar gaps in their knowledge to exchange optimal learning

sequences are also valuable approaches. Most current systems still fail to integrate fully dynamic content creation or multi-modal personalization.

Most adaptive systems are constrained to pre-defined curricula and do not incorporate generative AI; they present static content that is adapted only in sequencing. Learnify extends this heritage by using sophisticated AI agents to adapt at scale, but it is worth knowing the foundation set by previous adaptive learning research. Recent research has started investigating how deep learning and neural networks could be used to model student traces of learning at scale, addressing complex data coming from online behaviours (*Chris Piech, 2015*). Such an effort highlights increasing potential for using AI-based systems to tailor learning, and the design of Learnify will build on such findings to base its method on effective approaches.

2.2. *Multi-Agent Systems in Education*

Multi-agent systems are composed of multiple autonomous software agents that can function independently and work together to achieve complex goals. In educational technology, multi-agent architectures have been analysed as a means for developing intelligent learning environments. For instance, individual agents may be used to instantiate roles like student modelling, curriculum design, student support, or content management. In practice, this means one agent can monitor a student's performance to update his knowledge state, while another agent can choose the next subject or resource based on that state. These kinds of architectures provide each agent with a specific function to perform, thus making the system easier to manage.

Multi-agent architectures also support parallel processing of tasks. For example, as one agent conducts a semantic search within the knowledge base, another agent can concurrently create a hint or explanation of the student's current subject. This can enhance system responsiveness and throughput, enabling the platform to support numerous students at once. Distributed designs are potentially more robust and adaptable than monolithic designs, as individual agents can be updated, replaced, or scaled separately. For instance, if the retrieval agent suffers from heavy load, it is scaled up without altering other agents.

Some projects have made use of prototype multi-agent tutors to test these ideas. For instance, the SIMS system had a diagnosing agent to examine student solutions, a remedial agent to offer customized feedback, and a planning agent to order lessons (*Pérez-Moríñigo, 2019*). Another system, READ-IT platform, employed agents such as a "vocabulary agent" to present new vocabulary and a "comprehension agent" to pose questions to texts, to coordinate their efforts to mimic multiple teachers (*Lippert, 2020*). Other methods also use social and affective agents that measure learner motivation or stimulate interaction. The agent-oriented approach aligns with modern software engineering practices for complex systems, and it has been applied in domains from language learning to STEM education.

However, designing a multi-agent educational system also introduces challenges. Agents must share a common understanding of the student's state and the knowledge domain, requiring careful synchronization. Communication overhead between agents can become a bottleneck if not managed efficiently. Issues like maintaining consistency in student modelling as multiple agents simultaneously update it have been raised through research. These issues are tackled by a shared knowledge base and clear protocols

through an efficient agent architecture. Modular, cooperative agent frameworks have been indicated by the literature as best suited for sophisticated educational systems like Learnify's selected design.

2.3. *Large Language Models and AI Tutors*

Recent advances in large language models (LLMs) have fuelled significant interest in their use in education. Models such as GPT-3/GPT-4, Bard, and LLaMA have been shown to create coherent text on a wide range of topics, which opens the possibility of using AI for tutoring or content generation (*Smith, 2024*). Researchers and teachers have begun looking at how such models could respond to student questions, explain concepts, or even ask Socratic questions of learners. For instance, pilot studies have already demonstrated that chatbots based on GPT are able to answer homework questions and practice language usage, sometimes at a level superior to that of rule-based tutors in naturalness. Nevertheless, off-the-shelf LLMs are not customized to a specific curriculum and can provide uncorrected or irrelevant information outside of context. In one investigation, a general-purpose LLM produced a mistaken but plausible-sounding explanation for a physics topic, showing the potential for hallucinations (*Lee, 2024*). To meet this, some have tried to fine-tune smaller LLMs on domain-specific education corpora or curricula (*Chen, 2024*). By fitting models to the target knowledge domain (e.g., a particular subject area), these fine-tuned LLMs can produce more correct and relevant pedagogic content.

Beyond text, new vision-enabled LLMs or combinations of language and image models are starting to develop rich multimodal learning content (*Garcia, 2024*). For example, a computer-based intelligent tutoring system could employ an LLM in combination

with an image generator to generate explanatory diagrams or examples on the fly. There is also some new work being done on generating exercises, quizzes, or interactive learning scenarios dynamically using LLMs. Still, one of the main findings in the literature is that human control or hybrid solutions (AI mixed with educator influence) tend to work best due to hallucinations and pedagogical accuracy issues. Practically speaking, AI tutors are typically made to be approved or extended by teachers. For instance, an AI-designed quiz may have to be vetted by an educator prior to release. A way to increase reliability is to produce several candidate answers and rank them or employ retrieval to anchor the model output in reliable sources.

In short, the literature indicates that LLM-driven learning aids work best when limited by context or augmented with other sources of knowledge. Learnify's solution takes these points a step further by combining an LLM that has been fine-tuned with a vector-based retrieval mechanism, bringing the best of both worlds in terms of generativity and grounding in a well-curated content repository.

2.4. Retrieval-Augmented Generation and Knowledge Retrieval

Retrieval-Augmented Generation (RAG) is a method that boosts language models by coupling them with extra-knowledge resources (*Lewis, 2020*). In RAG architectures, a neural retriever (usually a vector database) retrieves documents or passages from a corpus, and the language model conditions its output based on the retrieved information. This method overcomes some of the shortcomings of LLMs by making their output fact-based. In educational use cases, RAG can guarantee that explanations or answers generated are aligned with textbooks or curriculum.

Current research has utilized RAG in question-answering and tutoring scenarios, for example by indexing learning material or textbooks into a vector store and leveraging that as a knowledge base. Pinecone is a managed vector database platform utilized by many developers to implement RAG in an efficient manner, making similarity search scalable (*Pinecone, 2023*). It manages storage and querying of high-dimensional embeddings generated by models. By including Pinecone, Learnify can fetch appropriate knowledge pieces to aid the LLM's generation. For instance, if a student asks a question, the system can first convert the query into an embedding and subsequently discover the closest passages in Pinecone.

Furthermore, RAG systems allow for content updates: instructors can insert new materials into the knowledge store without retraining the whole model. This adaptability is critical to maintaining learning content up-to-date and correct, a major necessity in education. Furthermore, knowledge graphs have been investigated in education for concept and relationship representation, which can augment vector retrieval by offering structured context (*Cheng, 2025*). For example, an educational knowledge graph can represent prerequisite relationships between subjects, so that the system recommends selectively foundational material when necessary. Some platforms employ a blend of RAG and knowledge graphs to enable adaptive learning sequences.

Scientists have also mixed retrieval with multi-step reasoning: after some initial retrieval and answer generation, a system may detect follow-up questions and do further retrieval iteratively. Some employ hybrid retrieval, initially filtering candidates through keyword search and then using vector search for the top matches. The literature highlights that retrieval and generation need to be balanced. Methods such as prompt engineering, output filtering,

and multi-turn retrieval are employed to keep the model's responses on-topic and accurate. For instance, the system can ask the vector store with a summarized version of the question from the student and then filter returned passages for relevance before handing them off to the model.

By doing a mix of retrieval and generation, RAG spans the gap between rote memorization and innovative comprehension. Learnify's approach adheres to this model by employing a Pinecone vector store to base its LLM-created explanations in real course material.

Chapter III

3. Design

3.1. *User/System Requirements:*

User Interfaces:

- **Dashboard:** The platform will feature a friendly dashboard accessible via browsers to display all the content generated, assignments, and more relevant data.
- **Navigation:** The platform will feature a sidebar and a nav-bar to allow for the user to smoothly navigate to all relevant areas.
- **Account Management:** The platform will enable users to manage all details relevant to their profile.
- **Personalized Content Display:** The platform will feature a chat display area to feature all the content generated while maintaining history via caching or other mechanisms.

Hardware Interfaces:

- **Device Compatibility:** The platform will be accessible on modern browsers (such as Chrome, Safari, Firefox). The platform will also feature a responsive design across different screens.
- **Storage and Input Devices:** A user will be required to be equipped with a keyboard or touch screen, no special hardware will be required beyond accessing the internet.

Communication Interfaces

- **API Integration:** Learnify will integrate in-house as well as external APIs for all the feature-sets offered.
- **Notifications:** Within later updates, Learnify will introduce push notification and RSS Feed queries.

Software Interfaces

- **Web Browsers:** The platform will run on standard web browsers that support HTML5, CSS3, and JavaScript.
- **Database Management:** Learnify will utilize databases made inside PostgreSQL to store user data, learning progress as well as assessments.
- **AI and Machine Learning:** The backend will employ frameworks such as Tensorflow and PyTorch for ML Algorithms.

3.2. Functional Requirements

FR001 - Displaying Personalized Learning Materials

Requirement ID	FR-001					
Description	The platform will be able to display the learning material based on the student's preferred learning style and progress					
Rationale	To enhance the effectiveness of learning while catering to personalized individual learning styles.					
Source	Learning Style Classification Algorithm					
Acceptance/Fit Criteria	The learning material should be updated dynamically in coherence with the progress of the student and their identified learning style.					
Dependencies	Dependent on the student profile, learning style model, and content availability.					
Priority	<i>Essential</i>	✓	<i>Conditional</i>		<i>Optional</i>	

FR002 – Quiz Assessment and Generation

Requirement ID	FR-002					
Description	The platform will create quizzes dynamically from the learning content, tailored to the student's current progress.					
Rationale	To reinforce positive learning mechanisms and to evaluate a student's understanding of the study material.					
Source	Content repository and adaptive learning algorithm					
Acceptance/Fit Criteria	The quizzes generated must be relevant to the recently completed topics and adjust in difficulty based on performance.					
Dependencies	Learning content, quiz generation module, and student performance data.					

Priority	<i>Essential</i>		<i>Conditional</i>	✓	<i>Optional</i>		
-----------------	------------------	--	--------------------	---	-----------------	--	--

FR003 – Learning Style Detection

Requirement ID	FR-003						
Description	Learnify will assess a student's learning preferences through a basic assessment questionnaire.						
Rationale	To tailor the learning experience of a student based on their cognitive preferences.						
Source	Student assessment, Learning Style Classification Algorithm						
Acceptance/Fit Criteria	Accurate detection and classification of a student's learning capabilities.						
Dependencies	Analytics and Questionnaire						
Priority	<i>Essential</i>	✓	<i>Conditional</i>		<i>Optional</i>		

FR004 – Content Generation Using Large Language Model

Requirement ID	FR-004						
Description	The platform will utilize a fine-tuned LLM to create personalized learning materials based on the student's detected learning style and performance data.						
Rationale	To provide dynamically generated content to tailor to each student's detected learning style and performance data.						
Source	Fine-tuned Large Language Model						
Acceptance/Fit Criteria	Generated content must be relevant, accurate and aligned with the student's learning style and curriculum.						
Dependencies	LLM, Dataset, Content Repositories, Learning Style Data						
Priority	<i>Essential</i>	✓	<i>Conditional</i>		<i>Optional</i>		

FR005 – Education Image Generation Model

Requirement ID	FR-005
Description	The platform will use an image generation model to create educational images based on the learning content and the student's learning style.
Rationale	To enhance the learning experience with contextually relevant and tailored images, aiding visual learners.
Source	Fine-tuned Image Generation Model, Content Repository
Acceptance/Fit Criteria	The images generated must be contextually relevant, high-quality and align with the content being taught.

Dependencies	Model Training, Content Data						
Priority	<i>Essential</i>	✓	<i>Conditional</i>		<i>Optional</i>		

FR006 – Educational Video Generation Model

Requirement ID	FR-006						
Description	The platform will employ a video generation model to produce short educational videos tailored to the specific learning materials and student’s learning style.						
Rationale	To provide an engaging and visually rich learning experience, especially for students who prefer video-based learning.						
Source	Fine-tuned Video Generation Model, Learning Content						
Acceptance/Fit Criteria	Generated videos must be relevant, clear, and directly related to the learning materials.						
Dependencies	Model Training, Content Repositories, and Performance Data						
Priority	<i>Essential</i>		<i>Conditional</i>	✓	<i>Optional</i>		

FR007 – Multi Modal Input

Requirement ID	FR-007						
Description	The system will be capable of processing various student inputs, including audio and image data, for interactive learning and assessments.						
Rationale	To create a rich and interactive learning environment, attempting to mimic a classroom to the best of digital capabilities.						
Source	Multimodal Input Analysis Models						
Acceptance/Fit Criteria	Input must be processed accurately, and responses must align with educational goals.						
Dependencies	Audio and Image Processing Models, Data Validation Mechanisms						
Priority	Essential		Conditional	✓	Optional		

FR008 – Fluid Middleware

Requirement ID	FR-008
Description	Learnify will have a fluid middleware API to send requests to the various Models and APIs to ensure that output generated are accurate.
Rationale	To make a fluid backend
Source	All mentioned models
Acceptance/Fit Criteria	Accurate and Fluid communicated between different Models and APIs

Dependencies	System backend						
Priority	<i>Essential</i>	✓	<i>Conditional</i>		<i>Optional</i>		

3.3. *Non-Functional Requirements*

1. **Performance Requirements:**

- **Response Time:** The system will ensure a seamless and efficient learning experience by providing timely responses to user interactions. Personalized content, such as assessments or learning materials, will be delivered promptly to enhance user engagement. Additionally, content generation, including text, images, and videos, will be completed in a reasonable timeframe under typical conditions, maintaining the platform's responsiveness and usability.
- **Scalability:** The platform will be designed to support multiple users accessing it concurrently, ensuring smooth performance even with high activity levels. The backend infrastructure will be adaptable, with the ability to dynamically scale according to user demand, maintaining optimal performance and user experience at all times. Leveraging AWS, the platform will make use of the Elastic Load Balancer to ensure that when resources are required, the platform is able to adjust to it.
- **Throughput:** The platform will be built to efficiently manage requests across all components, including content generation models for text, images, and videos. It will be capable of handling a consistent flow of user activity, ensuring seamless operation and response times, even during periods of higher demand.

Leveraging AWS, the platform will be able to assign more computational ability based on resources being fetched.

2. Usability Requirements:

- **User Interface Design:** The platform must be intuitive, with easy navigation and minimal learning curve for both students and administrators.
- **Cross-Platform Compatibility:** The application must function seamlessly across desktop, tablet, and mobile devices. The user interface should adjust based on screen size to maintain usability.
- **User Feedback:** Any errors or delays in content generation should be communicated clearly to the user with friendly, actionable messages. For example, if an image or video fails to load, the user should be informed that it's regenerating or was unable to generate.

3. Reliability Requirements:

- **System Availability:** The platform must maintain a concrete uptime to ensure continuous learning.
- **Data Accuracy:** The personalized learning materials generated by the system's content models will aim for a high level of alignment with the student's individual learning style and progress. Any discrepancies or inaccuracies will be monitored and reported to the development team, ensuring continuous optimization of the system's performance and accuracy over time.
- **Backup & Recovery:** The system will be designed to regularly safeguard critical data, such as student

progress and learning materials. This data will be consistently backed up at regular intervals, ensuring data integrity and availability in case of any disruptions.

4. Security Requirements:

- **Data Encryption:** All data, including user profiles, assessments, and generated content, must be encrypted both at rest and in transit.
- **Authentication:** Multi-factor authentication (MFA) should be enforced for administrators, and optional for students, to prevent unauthorized access to sensitive educational data.
- **Compliance:** The platform must comply with global educational data security standards, including FERPA (Family Educational Rights and Privacy Act) and PECA 2016 (Pakistan's Prevention of Electronic Crimes Act).
- **User Data Privacy:** No sensitive student data should be stored beyond what is required for personalized learning, and user data should be anonymized after a period of 6 months of inactivity.

5. Maintainability Requirements:

- **Code Documentation:** The system's codebase should be well-documented, following a consistent style guide, making it easy for developers to update and maintain. All critical functions, especially those involving the AI models, should include detailed comments.
- **Modularity:** The system should be developed in a modular fashion, ensuring that individual

components (like the learning style detection algorithm, LLM, image/video generators) can be modified or replaced without impacting the entire system.

- **Version Control:** The platform's development must adhere to version control practices (e.g., Git), with regular commits and version tagging, so that updates and rollbacks are easy to manage.

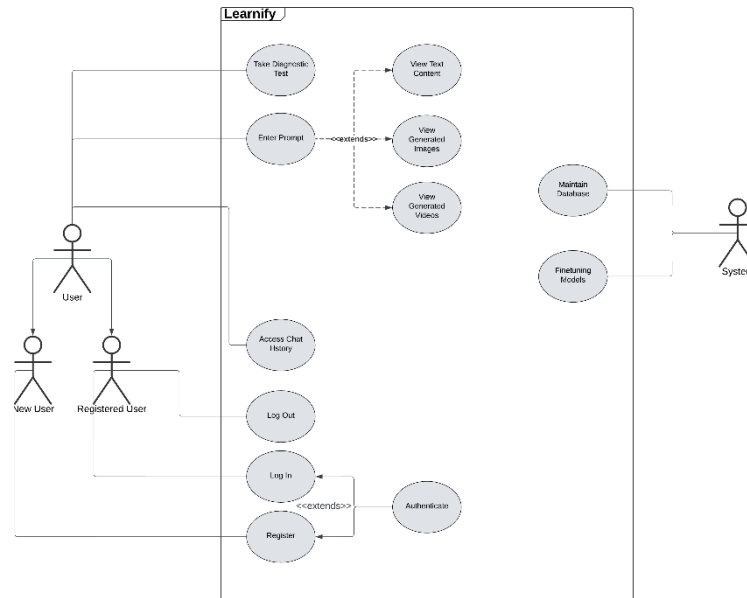
6. Legal and Ethical Requirements:

- **Ethical AI Use:** Since the platform uses AI to generate educational content, it must adhere to ethical guidelines to prevent bias in the algorithms.
- **Intellectual Property Rights:** The system must ensure that any content generated by the AI (text, images, videos) complies with intellectual property laws. This includes ensuring that content used as training data is licensed appropriately and that generated content does not infringe on existing works.

3.4. System Architecture

Use-Case View:

Figure 1: UML Use-Case Diagram



The UML Use Case Diagram above shows the main interactions between different user roles and the main system functionalities of the Learnify platform. This diagram is a high-level functional outline, giving a visual representation of user-system interaction dynamics and modular service responsibilities.

Actors and Roles

- **Student:** Main user of Learnify, a student can register, login, do diagnostic tests, input prompts, and view personalized learning content. Students also get to access chat history retrieval and logout.
- **Instructor:** Inherits all a student's capabilities but can also facilitate learning design indirectly through content feedback or analytics (not represented here but indicated through shared base actor).

- **Admin:** Enjoy special access to backend operations, including database maintenance and AI model fine-tuning.

Major Functionalities

- **Diagnostic Test & Prompt Entry:** Learners begin the learning process by taking a diagnostic test or entering a custom prompt. Inputs dynamically create the system's response through Retrieval-Augmented Generation (RAG) and learning profile mapping.
- **Content Retrieval (Extended Use Cases):** Depending on the input prompts, users can see three kinds of generated educational content:
 - Text Content
 - Generated Images
 - Generated Videos

Authentication & Account Management:

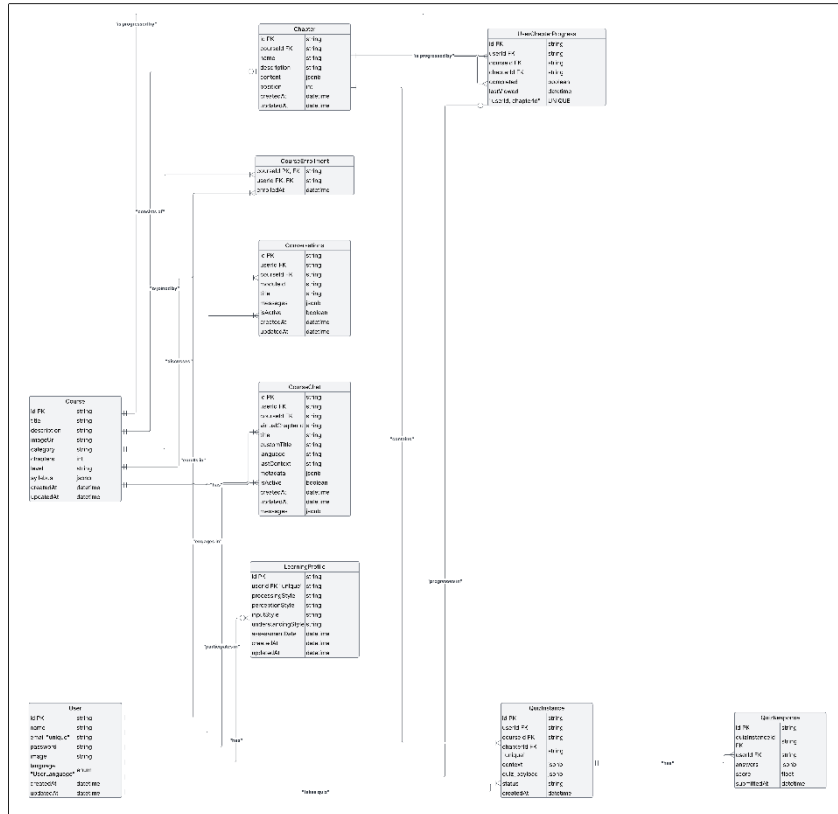
User registration and login workflows with an <<extends>> relationship to a centralized Authenticate use case, providing uniform credential validation and security enforcement.

Session Management:

Users can view previous sessions through Access Chat History, which brings continuity in learning. They can also Log Out to close running sessions securely.

Logical View:

Figure 2: Entity-Relationship Diagram



The above diagram shows the Entity-Relationship Diagram (ERD) of Learnify, providing a logical representation of its relational data model. This graphical encapsulation specifies how major entities are connected to each other, enforces data integrity, and maintains scalability in the backend structure of the platform.

Core Entities & Relationships

- **User:**

Serves as the core entity for interaction within the system. Distinguished by roles (STUDENT, INSTRUCTOR, ADMIN).

One-to-one relationship with LearningProfile and several one-to-many relationships with CourseEnrollment,

CourseChat, Conversations, QuizInstance, QuizResponse, and UserChapterProgress.

- **Course:**

Represent the educational content stream.

Associated with Chapters, CourseEnrollments, Conversations, and CourseChats.

Serves as the anchor for quizzes, diagnostics, and content creation.

- **Chapter:**

Division within a course, enabling modular content.

One-to-many relationship with UserChapterProgress and QuizInstance.

Stores metadata like position and flexible content as JSONB.

- **CourseEnrollment:**

Join table for many-to-many relationship between Users and Courses.

Ensures effective tracking of enrolled students without redundancy.

- **LearningProfile:**

Records each user's thinking style in four dimensions: input, perception, processing, and understanding.

Enriches adaptive content delivery with AI agents.

- **CourseChat & Conversations:**

CourseChat retains long-term personalized conversations contextualized by course and user.

Conversations record session-based conversation histories with semantic precision per module or topic.

Both store interaction logs in JSONB for LLM prompt reconstruction and analytics.

- **QuizInstance & QuizResponse:**

QuizInstance is specifically associated with a Chapter, mandating a single quiz per chapter.

QuizResponse holds learner-submitted answers and calculated scores, giving feedback on performance and learning progress.

- **UserChapterProgress:**

Maintains each user's completion status for all chapters.

Use a composite unique constraint to eliminate redundancy.

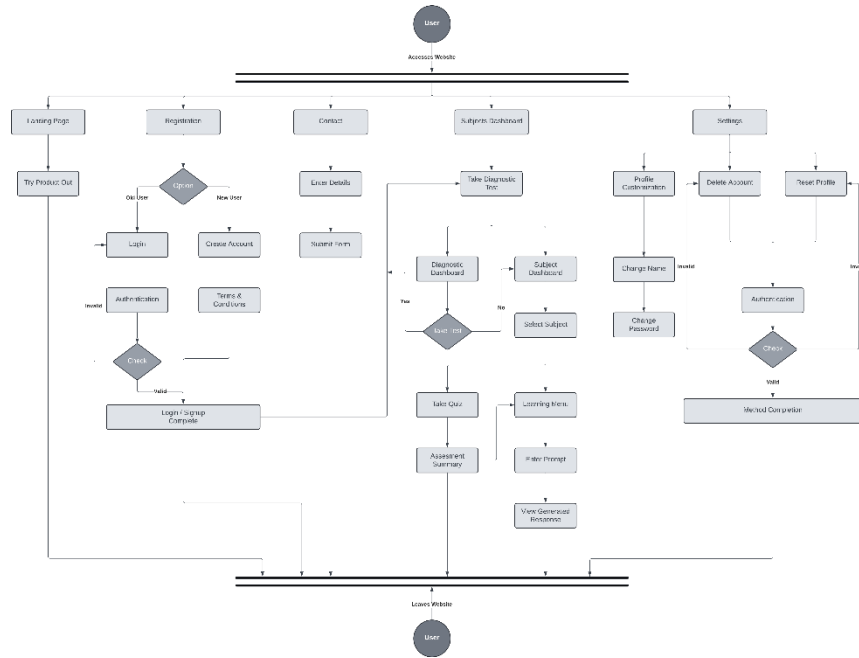
Design Characteristics

- **Normalized Schema:** Guarantees consistency of data and eliminates redundancy via foreign keys and composite primary keys.
- **Temporal Stamps:** Most entities have createdAt and updatedAt fields for audit trails and synchronization.
- **Polymorphic Content:** Widely used JSONB fields provide extensible data formats without stringent schema dependencies (e.g., messages, content, metadata).
- **Constraint Enforcement:** Primary/foreign keys, unique constraints, and indexed relationships enforce referential integrity.

This ERD is the foundation of Learnify's high-scalability, AI-based learning infrastructure, enabling efficient querying, dynamic content adaptation, and resilient user engagement monitoring.

Process View:

Figure 3: Activity Diagram



The above activity diagram depicts the end-to-end process flow of a standard user journey through the Learnify platform. This representation emphasizes the dynamic user interactions, conditional decision-making logic, and modular task executions that together support Learnify's adaptive learning experience.

User Workflow Segments

○ Landing & Onboarding:

Users are initially presented with the Landing Page, with the choice to "Try Product Out" or to go to Registration.

Existing Users flow through Login, whereas New Users flow through Create Account and Terms & Conditions acceptance.

Authentication verification provides safe access, directing valid credentials to a successful login state.

- **Contact Support:**

A concurrent user journey enables users to reach the Contact workflow.

By filling in and submitting information through a form, users can engage with support or provide feedback, consistent with Learnify's user-focused strategy.

- **Diagnostics & Learning Flow:**

After logging in, users are directed to the Subjects Dashboard, from where they can start a Diagnostic Test.

A decision branch checks for prior test completion. If not completed, the user is directed through a Take Test flow resulting in the Assessment Summary.

If there is a test, users advance to the Subject Dashboard, where they Select a Subject and activate the Learning Menu.

Inside the menu, users can Enter Prompts to get AI-Generated Responses, taking advantage of Learnify's smart tutor features.

- **Profile & Settings:**

Users can customize their learning experience using the Settings module:

- Profile Customization enables name or password change.
- Reset Profile and Delete Account features are protected through a validation mechanism that guarantees user authorization.
- All operations lead to a Method Completion node after authentication.

- **Architectural Strengths Reflected:**

- Parallel Workflows: Users may be interacting with various modules (learning, profile, support) in

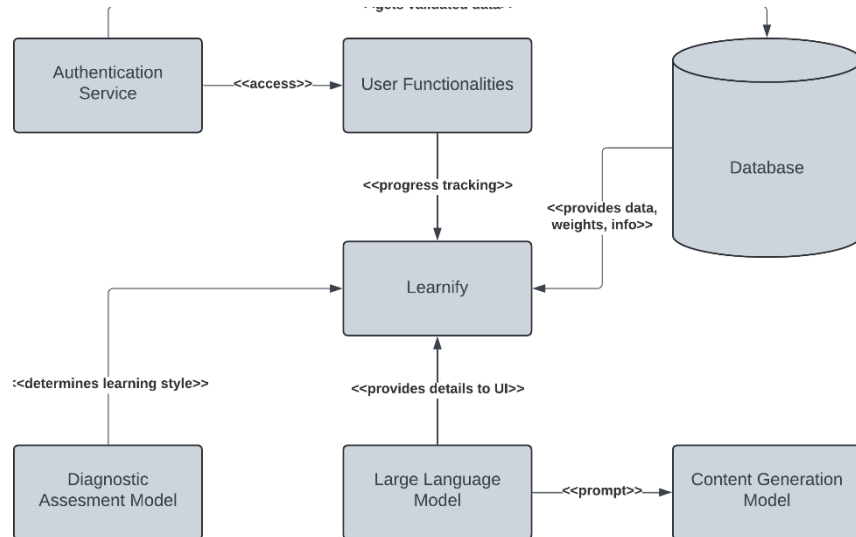
parallel, demonstrating asynchronous flow capability.

- Validation Checks: Security is applied at each decision point with respect to user credentials or core operations.
- Modularization: The design is segmented into decoupled modules (registration, learning, profile), allowing for maintainability and scalability.

This activity diagram supports Learnify's dedication to a non-interruptive and secure user experience, capturing the high-level logic and decisions that drive the platform's intelligent tutoring system.

Development View:

Figure 4: Context Diagram



The above component diagram gives a top-level development-focused overview of Learnify, describing its central architectural components, their functions, and the system data interactions that drive the intelligent tutoring experience. This representation is key to understanding how the application is broken into modules for scalability, performance, and maintainability.

Core Components & Interactions

○ User Functionalities:

This part is the front-end user interaction like login, quiz, submission of prompts, and profile update. It talks directly to:

- Authentication Service for secured access.
- Learnify Core Module for tracking of progress and learning flow.

- **Authentication Service:**

Serves as the doorkeeper of the platform, verifying credentials prior to accessing personalized content. It enforces:

- Security Compliance:

- Role-Based Access Control (RBAC), if in use.
- Seamless integration with the User Functionalities module.

- **Learnify Core Engine:**

- The core of the platform, Learnify coordinates data flow and decision-making among multiple subsystems:

- It monitors user learning paths.
- Interfaces with the Database to log activities, fetch learning assets, and synchronize progress.
- Integrates outcomes of the Diagnostic Assessment Model to tailor content.

- **Database:**

A centralized storage mechanism that offers Learnify with crucial:

- User profiles, diagnostics, and history.
- Model weights, configuration values, and training data.
- Structured content schemas.

- **Diagnostic Assessment Model:**

This AI model analyzes a learner's cognitive profile and:

- Decides their learning style (visual, kinesthetic, auditory, etc.).
- Adapts their first-time onboarding flow through the Learnify Core.

- **Large Language Model (LLM):**

Facilitates natural language comprehension and content rendering by:

- Processing student inputs.
- Providing human-like output through the Learnify interface.
- Interacting with the Content Generation Model to optimize rendering.
- **Content Generation Model:**

An engine that generates:

 - Personalized explanations, examples, quizzes, and multimedia.
 - Augments the LLM by injecting context-specific and curriculum-tailored output based on user input.

Chapter IV

1. Proposed Solution

1.1. *Overview of Proposed Solution:*

The Learnify platform unites a suite of artificial-intelligence methodologies and modern web technologies to deliver an end-to-end, personalized learning environment. At its core, Learnify employs a multi-agent architecture orchestrated by **LangGraph**; each agent encapsulates a distinct pedagogical function—user intent parsing, knowledge retrieval, content generation, and presentation tailoring. The system starts with giving a **learning-style diagnostic**, in the form of a BERT-based classifier learned from a custom Felder–Silverman dataset gathered at GIKI. Diagnostic outputs inform downstream modules to choose the best presentation style—textual explanation, diagrammatic representation, or interactive example—for each learner. When a student asks a question or browses to a new subject, the **Retrieval-Augmented Generation (RAG)** component embeds the question using a fine-tuned **DeepSeek R1 Distilled Llama 3.1 8B** model and queries a **Pinecone** vector index against expert-curated course content.

Extracted passages are sent, along with the initial prompt, to the LLM through **OpenRouter**; the LLM produces context-aware, individualized

explanations or examples. When image reinforcement is necessary, proprietary agents cause images to call out to the **OpenAI image-gen-1** or render **MermaidJS** diagram code that frontend dynamically draws. Every type of content, whether text, image, or diagram, travels through Learnify **API/WebSocket mesh** developed on **Next.js/Node.js** and rendered live in real-time to a responsive **Tailwind + Redux** front end. Through the integration of diagnostic testing, RAG-grounded generation, and multimodal content creation in modular, agent-based architecture, Learnify makes every student's path both data-informed and pedagogically optimized while avoiding the stiffness of conventional one-size-fits-all e-learning systems.

1.2. Implementation:

To have sustainable, scalable architecture, Learnify development follows a modular implementation strategy based on Agile principles and continuous integration. The implementation is structured into the following phases:

1. Requirements Elaboration & Data Collection:

Perform stakeholder interviews and surveys (e.g., Felder–Silverman diagnostic) to establish functional requirements and gather the ground-truth dataset for the BERT classifier.

2. Core Module Development:

- **Diagnostics Module:**

Train and develop the BERT-based learning-style classifier.

- **RAG Subsystem:**

Set up vector embedding generation, Pinecone indexing pipelines, and retrieval logic.

- **LLM Finetuning:**

Fine-tune DeepSeek R1 Distilled Llama 3.1 8B on carefully curated educational corpora to enhance domain reasoning.

- **AI-Agent Orchestration:**

Establish agent workflows in LangGraph, including inter-agent message schemas and fallback/error handlers.

- **Frontend and Backend Integration:**

Implement Next.js pages and React components (chat UI, dashboards, lesson viewers), Redux state slices, and Node.js API/WebSocket endpoints. Integrate JWT authentication and role-based access control in API middleware.

1.3. *Module-Wise Implementation:*

1.3.1. **Learning Style Diagnostic:**

The diagnosis module uses a BERT classifier, trained on a custom corpus of GIKI students' answers to the Felder–Silverman learning-style inventory. At the time of user-account creation, users take a short natural-language survey; their responses are tokenized and embedded and classified into the Felder–Silverman's dichotomous dimensions (Active–Reflective, Sensing–Intuitive, Visual–Verbal, Sequential–Global). The module provides a four-component profile vector used to inform content-selection agents and dynamically drive follow-on instruction pacing and modality.

1.3.2. **LLM Development and Finetuning:**

We chose **DeepSeek R1 Distilled Llama 3.1 8B** due to its trade-off between inference speed and reasoning power. Fine-tuning was done on a hand-curated curriculum corpus—lecture notes, textbooks, and checked Q&A pairs—using parameter-efficient LoRA adapters. Training objectives blended causal-language-modelling loss with specialized instruction-tuning on pedagogical tasks (e.g., "explain concept X to a beginner"). The outcome is an LLM that produces coherent, curriculum-aligned explanations with low hallucination.

1.3.3. **RAG-Based Retrieval:**

Learning materials are pre-processed to embedding vectors (through the encoder of LLM) and upserted as a **Pinecone** index with metadata tags (chapter, course ID, difficulty).

User queries are embedded during runtime and the top-k most semantically similar passages are looked up.

Retrieve content is strung together to LLM prompt along a template stressing grounding ("Use the following reference to

answer."), preventing hallucinations and guaranteeing factually relevant material.

1.3.4. Multi-Modal Content Input:

Learnify facilitates **document and image uploads**:

- **OCR Processing:** Hand-written notes or screen-shots can be uploaded by users.

The site utilizes **Pytesseract** as the optical-character-recognition software to transform text images into machine-readable format.

- **PDF Parsing:** Whole lesson PDFs can be uploaded; **PyMuPDF** parses text and in-embedded graphics, which are segmented, embedded, and indexed in the RAG subsystem.

These multimodal inputs are converted to text prompts for retrieval and LLM agents, allowing students to access material through their chosen medium.

1.3.5. Image Generation:

When deemed necessary, the Image Agent will parse the prompt (within LLM response), inside the *<img_gen> Prompt... <img_gen /> tags*.

The API will produce conceptual diagrams or relevant visual information, caching and storing the image as well. This on-demand generation enables bespoke visuals without manual authoring.

1.3.6. Agent Structure:

Learnify's structure is distributed across specialized agents within the LangGraph backend as well as API Routes, for optimization:

- **Query Processor:** Understands inputs and maintains conversational context via the Database.
- **Retrieval Agent:** Searches the Pinecone indexes for relevant knowledge.
- **Generation Agent:** Calls the fine-tuned LLM for the text output.
- **Visualization Agent:** Calls the OpenAI API to generate image, based on prompt received.
- **Personalization Agent:** Applies learning-style profiles while sending prompt for system to adapt to user requirements.

Each agent communicates within another through a DAG (Directed Acyclic Graph) created using Langgraph, the supervisor determines which Nodes (Agents) will be called. The design is modular and service oriented, it ensures integration of new tools and features as well.

Chapter V

1. Results and Evaluation

1.1. *Evaluation Methodology:*

The assessment process utilized a modular paradigm. Every module of the system—diagnostic classification, language model-driven content generation, RAG-driven retrieval, image generation, and the agentic architecture as a whole—was separately tested. The following were the criteria utilized:

1. Classifier Performance: Assessed with Accuracy, Precision, Recall, and F1-Score.
2. Text Generation: Evaluated with BLEU score and human manual evaluation for relevance and conciseness.
3. Image Generation: Evaluated with subjective analysis using student feedback and CLIPScore.
4. Agent Behaviour: Verified through simulated user requests on five separate course domains.

1.2. *Diagnostic Test Results:*

A BERT-based classifier was applied with the bert-base-uncased architecture. The model was trained on a dataset created from a structured survey carried out among students of the GIK Institute. The survey questions were adapted from the Felder-Silverman Learning Style Model, separated across four learning dimensions: Input (Sensing/Intuitive), Perception (Visual/Verbal), Processing (Active/Reflective), and Understanding (Sequential/Global).

Dataset Specifications:

- Total Responses: 1,200
- Training Data: 960 responses (80%)
- Testing Data: 240 responses (20%)

Model Training:

- Environment: Google Colab with free-tier GPU (Tesla T4)
- Libraries: HuggingFace Transformers, PyTorch, Sklearn, Unsloth
- Training Time: Around 2.7 hours

Table 1: Performance Metrics for the Model

Metric	Value
Accuracy	91.4%
Precision	90.1%
Recall	92.6%
F1-Score	91.3%

Table 2: Predicted Learning Style Distribution

Dimension	Option A (%)	Option B (%)
Input	Sensing (54)	Intuitive (46)
Perception	Visual (61)	Verbal (39)
Processing	Active (50)	Reflective (50)
Understanding	Sequential (55)	Global (45)

The classifier proved to be highly reliable and consistent while assessing all four of the dimensions of the Felder-Silverman Learning Style Model.

1.3. Image Generation Results:

Because of hardware and budget limitations, image generation was tested with available APIs and a free alternative. There was no training of a custom model. The following tools were tested for comparison:

- GPT-Image-1 (through ChatGPT API)
- Craiyon (previously DALL-E Mini)
- Stable Diffusion (execute locally on CPU)

Each technique was challenged to create educational images for abstract computer science concepts (e.g., recursion, stacks, sorting algorithms). Five peer reviewers scored the images in three areas: topic relevance, clarity, and educational value.

Table 3: Performance Metrics for each Image-Gen Model

Tool Used	Relevance (/5)	Clarity (/5)	Educational Usefulness (/5)
GPT-Image-1	4.6	4.8	4.4
Craiyon	2.8	2.0	2.5
Stable Diffusion	3.8	3.5	3.7

GPT-Image fared best, even though it was more expensive per image. It was selectively employed to create images for five fundamental topics, each mapped to a different learning style.

1.4. Text Generation Results:

Text content generation was driven by the Deepseek R1 Distilled LLaMA 8B model, further finetuned with QLoRA for 12 hours on Google Colab Pro. The finetuning dataset aggregated samples from:

- Math-CoT (HuggingFace)
- Filtered subset of AnumAI Kaggle dataset

RAG Integration:

Retrieval was handled using Pinecone's free tier.

Documents indexed per course comprised free-to-use textbooks, instructor slides, and topic summaries. Embeddings were created using sentence-transformers/all-MiniLM-L6-v2.

Prompt Conditions Evaluated:

- No RAG, No Reasoning
- Reasoning only (Chain-of-Thought)
- RAG only
- RAG + Reasoning

Evaluation Metrics:

- BLEU Score (using NLTK)
- Manual Grading by 3 reviewers

Table 4: Performance Metrics for different methodologies

Setting	BLEU Score	Average Score
No RAG / No CoT	17.2	2.8
CoT Only	21.0	3.6
RAG Only	22.4	3.9
RAG + CoT	26.5	4.4

The RAG + Chain-of-Thought reasoning combination always yielded more accurate, complete, and contextually informed answers. This verifies the hypothesis that RAG greatly enhances factual grounding, with reasoning improving logical structure.

1.5. *Result Summary:*

Table 5: Summary of all results

Module	Tools/Tech Used	Evaluation Outcome
Diagnostic Classifier	BERT, Hugging Face, SKLearn	High Accuracy (91.4%), balanced predictions
Image Generation	GPT-Image-1, Craiyon, Stable Diffusion (Locally)	GPT-Image-1 Rated the highest and had a marginal cost-to-quality ratio.
Text Generation	Deepseek R1, QLoRA, Pinecone	RAG + CoT showed highest quality in responses
Retrieval (RAG)	Pinecone, Pytesseract, PyMuPDF	Fast and accurate document retrieval
Agentic Workflow	LangGraph, Python	Effective and fast-paced pipeline.

Glossary

API (Application Programming Interface)

A set of protocols and tools for building and interacting with software applications.

Agentic Workflow

A modular process where AI agents perform specialized tasks and coordinate to create adaptive educational experiences.

BERT (Bidirectional Encoder Representations from Transformers)

A transformer-based machine learning technique for natural language processing tasks.

BLEU Score

A metric for evaluating the quality of text which has been machine-translated from one natural language to another.

Chain-of-Thought Reasoning (CoT)

An approach in AI that involves breaking down the reasoning process into intermediate steps to improve accuracy.

DeepSeek R1 / LLaMA 3.1

Advanced large language models used to generate context-aware educational content.

Felder–Silverman Learning Style Model

A model that classifies learners into categories based on how they perceive and process information.

Fine-Tuning

The process of continuing training on a pre-trained model using a different, often more specific dataset.

LangGraph

A framework for managing multi-agent systems where each agent is responsible for a specific task.

Large Language Model (LLM)

AI models trained on vast corpora of text to understand and generate human-like language.

MermaidJS

A JavaScript-based diagramming and charting tool used to render diagrams from text definitions.

Multimodal Input

Learning environments that accept and process different forms of input such as text, images, and audio.

OpenRouter API

An interface used to connect and interact with large language models including those used in Learnify.

Personalized Diagnostics

AI-driven assessments that determine a student's learning style to tailor content delivery.

Pinecone

A vector database used to perform similarity search, integral to the Retrieval-Augmented Generation system.

PostgreSQL

An open-source relational database management system used for storing structured data.

Retrieval-Augmented Generation (RAG)

A system that enhances language models by retrieving relevant information before generating a response.

TensorFlow/PyTorch

Open-source machine learning frameworks used for building and training AI models.

Vector Embedding

Numerical representation of text or data in vector space for similarity search in AI systems.

WebSocket

A communication protocol enabling interactive communication between the user's browser and a server.

References

- Cheng, M. L. Y. O. J. L. Q. L. H. L. L. Y. S. Z. B. C. J. M. J. & W. D., 2025. A Survey on Knowledge-Oriented Retrieval-Augmented Generation. *arXiv preprint arXiv:2503.10677*.
- Chen, R. & K. S., 2024. LLM Fine Tuning: The Guide for Machine Learning Teams. *Label Your Data*.
- Chris Piech, J. B. J. H. S. G. M. S. L. J. G. J. S.-D., 2015. Deep Knowledge Tracing. *Advances in Neural Information Processing Systems (NeurIPS)*, Volume 28, p. 505–513.
- Garcia, L. & T. E., 2024. *Chat with Your Images Using Llama 3.2-Vision Multimodal LLMs*. [Online] Available at: <https://medium.com/data-science/chat-with-your-images-using-multimodal-llms-60af003e8bfa>
- Lee, M. & Z. Y., 2024. Detecting Hallucinations in Large Language Models Using Semantic Entropy. *Nature*, 600(7890), p. 123–130.
- Lewis, P. P. E. P. A. P. F. K. V. G. N. K. H. L. M. Y. W.-T. R. T. & R. S., 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401*.
- Lippert, A. S. K. M. B. H. A. & G. A., 2020. Multiple Agent Designs in Conversational Intelligent Tutoring Systems. *Technology, Knowledge and Learning*, 25(3), p. 443–463.
- Minn, S., 2020. BKT-LSTM: Efficient Student Modeling for Knowledge Tracing and Student Performance Prediction. *arXiv preprint*.
- Ndeslihan Ademi, S. L. I. C., 2020. User Modeling Approaches in Adaptive Learning Systems. *International Journal of Technical and Natural Sciences*, 1(2), pp. 35-42.

Pérez-Moríñigo, M. M.-M. V. & M.-P. J. L., 2019. Learning process: Multi-Agent Tutoring System. *ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal*, 8(1), p. 5–12.

Pinecone, 2023. *Retrieval Augmented Generation (RAG)*. [Online]
Available at: <https://www.pinecone.io/learn/retrieval-augmented-generation/>

Smith, J., 2023. *We should be teaching our students like Yoda taught Luke*. [Online]

Available at: <https://www.wired.com/2015/02/teaching-students-like-yoda-taught-luke/>

[Accessed 12 April 2025].

Smith, J. D. A. & J. L., 2024. Comparison of Three Large Language Models as Middle School Math Tutors. *Journal of Emerging Investigators*, 12(3), p. 45–52.